BACKEND ENGINEERING
FROM FIRST PRINCIPLES

Lecture 23

Concurrency and Parallelism

I/O-bound vs CPU-bound work • Threads • Event loops • Async/await • Goroutines • Race conditions

Detailed study notes | GitHub-ready companion Markdown

How to use these notes

Start with the distinction between I/O-bound and CPU-bound work, then compare concurrency with parallelism. The implementation models are easier to understand once that foundation is clear.

Important qualification

The transcript uses simplified mental models and illustrative numbers. Real runtimes and operating systems have additional details; the notes flag key nuances where needed.

1. Why concurrency matters

Core idea

A backend server receives many requests, and most requests spend substantial time waiting for external resources such as databases, caches, APIs, files, or network sockets. If the server handles only one request synchronously, it can sit idle while that request waits, leaving other requests waiting too.

The lecture's illustrative numbers

A simple local database query may take roughly 1–2 ms; a different availability zone may take 20–30 ms; a distant region may take 90–100 ms. A more involved API might perform five network operations at about 50 ms each (about 250 ms waiting) but use only about 10 ms of CPU time. These are teaching examples, not universal benchmarks.

Why it matters

Concurrency lets the process use otherwise-idle CPU time to make progress on other work while one task waits for I/O. This improves resource utilization and allows many requests to remain in progress at the same time.

2. I/O-bound vs CPU-bound work

I/O-bound

Work whose progress depends mainly on an external operation completing: database queries, network calls, file reads/writes, logging to output, and waiting for external services. The CPU cannot complete the external operation by executing more instructions.

CPU-bound

Work whose time is dominated by computation on the CPU: image manipulation, video encoding, large numeric calculations, intensive transformations, and some cryptographic operations.

Backend implication

Typical API services are often I/O-heavy, so concurrency is essential. CPU-heavy work may need parallel execution across cores or offloading to workers/processes. A service can contain both kinds of work.

3. Concurrency vs parallelism

Concurrency

A program is structured so multiple tasks can make progress during overlapping periods. A task can run, pause while waiting, and resume later. On a single CPU core, only one instruction stream executes at an instant, but the system can interleave tasks.

Parallelism

Multiple tasks execute at the same instant on different CPU cores. It requires hardware capacity for simultaneous execution.

Memory aid

Concurrency is about dealing with many things at once; parallelism is about doing multiple things at once.

Single-core timeline

Request A uses the CPU for parsing/validation, starts a database request, and yields while waiting. Request B then uses the CPU. When A's I/O completes, A becomes runnable and later gets CPU time again. Both requests are in progress even though one core executes only one at a time.

Multi-core timeline

With two available cores, A and B may execute CPU instructions simultaneously. That is parallelism.

4. Scheduling and threads

Thread

An operating-system-managed unit of execution. A thread has execution state, a stack for function calls/local data, and an instruction pointer/program counter identifying where execution should resume.

OS scheduler

Chooses which runnable thread gets CPU time. In preemptive scheduling, the OS can pause a thread after a time slice, save its state, and run another thread.

Blocking I/O

When a thread waits on a blocking operation such as reading a database socket, it cannot make CPU progress on that operation. The OS marks it blocked and can schedule another runnable thread. When I/O completes, the thread becomes runnable; it is not necessarily scheduled immediately.

Processes vs threads

Separate processes have isolated address spaces. Threads within the same process share heap/global memory, while each thread has its own execution stack and state. Shared memory makes communication fast but introduces synchronization hazards.

Parallel threads

On multiple cores, separate threads in one process can execute simultaneously, which can speed up CPU-bound work.

5. Native thread overhead

Memory

Each native thread needs a stack and OS bookkeeping. The lecture gives Linux stack reservations around several MB as an illustrative scale; actual virtual/committed memory depends on OS, configuration, and runtime. Large numbers of threads can consume substantial memory.

Creation cost

Creating a native thread requires interaction with the kernel and setup of its stack and scheduling state. This is more expensive than creating a lightweight user-space task.

Context-switch cost

Switching threads requires saving/restoring CPU registers and scheduler state. It is useful overhead, not application work; excessive switching can reduce throughput and increase latency.

Practical lesson

A thread-per-request design can be simple, but an unbounded number of native threads is costly. Use bounded pools or lighter concurrency abstractions where appropriate.

6. Event-loop concurrency

Basic model

An event-loop architecture commonly uses one main execution thread to run short callbacks/tasks. When a task starts asynchronous I/O, it registers what should happen when the operation completes and yields control instead of blocking the event-loop thread.

Operating-system support

The runtime uses OS facilities to monitor many sockets/descriptors efficiently. The lecture mentions epoll on Linux and kqueue on macOS; Windows has its own I/O notification mechanisms.

Lifecycle

Start request A → initiate I/O and register continuation → yield → process request B and other ready work → OS reports A's socket ready → run A's callback/continuation → send response.

Callbacks

A callback is a function supplied now to be executed later when an operation completes. Nested callbacks can become difficult to read and maintain.

Async/await

In JavaScript, async/await gives asynchronous code a sequential-looking syntax. `await` suspends the current async function until the awaited promise settles, allowing the event loop to run other work. It does not block the entire event-loop thread when used with genuinely asynchronous I/O.

Important limitation

Long synchronous CPU work blocks the event loop. While that code runs, the loop cannot process other callbacks or requests. CPU-heavy work should be broken up, moved to worker threads/processes, or handled by a separate worker service.

7. Async functions as state machines

Mental model

The lecture explains that an async function can be understood as a state machine. At each await point, the runtime preserves enough state to continue later.

Example flow

State 0 starts a user query and suspends. When the query completes, continuation moves to state 1, stores the user, and starts an orders query. When that completes, continuation moves to state 2 and returns the combined result.

Why async and await work together

The async function is transformed/represented so its execution can suspend and resume. Await is valid only in the appropriate asynchronous context (with language-specific exceptions such as top-level await).

Why blocking is harmful

If the event-loop thread is stuck in synchronous work, the continuation cannot run and other event-loop tasks are delayed.

8. Go goroutines and the Go runtime scheduler

Goroutines

Go exposes lightweight concurrent execution units called goroutines. The Go HTTP server commonly runs handlers in separate goroutines rather than creating one native OS thread per request.

M:N scheduling

The Go runtime multiplexes many goroutines over a smaller number of OS threads. The runtime scheduler decides which goroutine runs on which available thread.

I/O wait

When a goroutine waits on supported network I/O, the runtime can park it and let an OS thread run another runnable goroutine. When the I/O is ready, the goroutine can become runnable again.

GOMAXPROCS nuance

GOMAXPROCS controls how many OS threads may execute Go code simultaneously (the number of logical processors available to Go code), not a hard cap on all OS threads the process can ever create. The lecture’s simplified 'one thread per core' picture is a mental model.

Why lightweight

Many goroutines can be created because their stacks start small and grow as needed, and scheduling is handled by the Go runtime rather than requiring one OS thread per goroutine.

Do not overgeneralize

Goroutines are lightweight, not free. Excessive goroutines, unbounded queues, memory retention, or blocking system calls can still exhaust resources.

9. Race conditions and shared state

Shared memory

Threads in one process can read and write the same heap objects/global variables. Concurrent access without a safe coordination strategy can produce incorrect results.

Lost update example

A counter starts at 0. Thread A reads 0; thread B also reads 0; both increment their local register to 1; A writes 1; B writes 1. The final result is 1 even though two increments were requested. This is a lost update race.

Async race example

A balance starts at 100. Two withdrawals of 100 both check that the balance is sufficient before either asynchronous operation completes. Both pass the check. Each later subtracts 100, leaving -100. A single-threaded event loop can still have race conditions across await points because operations interleave.

Key insight

Single-threaded does not mean race-free. An async function can observe shared state, yield at await, and resume after another task has changed that state.

10. Preventing race conditions

Locks / mutexes

A lock protects a critical section so only one participating execution enters it at a time. Acquire before the shared update and release reliably afterward. Keep critical sections short and use consistent lock ordering to reduce deadlocks.

Atomic operations

For simple counters/flags, atomic read-modify-write operations can provide safe updates without a larger lock (where supported and correctly used).

Message passing / channels

Instead of allowing many goroutines to mutate the same variable, send messages to one goroutine that owns and updates the state. Go channels support this style.

Database coordination

For balances, inventory, and other business-critical state, enforce invariants at the database boundary using transactions, conditional updates, row locks, or constraints as appropriate. In-memory locks alone do not coordinate multiple processes/servers.

Design principle

Reduce shared mutable state where possible. Define ownership, synchronization, and consistency boundaries explicitly.

11. Choosing a concurrency model

I/O-heavy API server

Use asynchronous I/O/event loops, goroutines, virtual threads, or a bounded thread pool depending on the language/runtime and libraries. Prioritize avoiding unnecessary blocking and controlling resource use.

CPU-heavy workload

Use parallelism across cores—worker threads, processes, or a dedicated job system—because simply interleaving tasks on one core does not make CPU computation finish faster.

Mixed workload

Keep request-handling paths responsive; offload expensive CPU tasks to bounded workers and apply backpressure/queue limits.

Do not choose by slogan

The right model depends on runtime behavior, library support, workload profile, memory, latency, and operational constraints. Measure the real bottleneck.

12. Practical backend checklist

Workload

Identify whether each expensive operation is I/O-bound or CPU-bound. Measure time spent waiting versus computing.

I/O

Use non-blocking/asynchronous APIs where supported. Avoid accidentally calling blocking libraries on an event-loop thread.

CPU

Move long CPU tasks off the event loop; bound worker concurrency and queue size.

Threads/tasks

Avoid unbounded thread or goroutine creation. Use pools, limits, cancellation, and timeouts.

Shared state

Look for global variables, shared maps, counters, balances, caches, and read-modify-write sequences that cross await points.

Correctness

Use locks, atomics, channels/ownership, transactions, or constraints as appropriate. Test concurrent paths and failure/retry behavior.

Operations

Monitor request latency percentiles, throughput, CPU, memory, thread/goroutine counts, event-loop lag, queue depth, and downstream wait time.

13. Interview questions and answers

What is concurrency?

Structuring a program so multiple tasks can make progress over overlapping periods, even if a single core interleaves them.

What is parallelism?

Executing multiple operations at the same instant on multiple processing cores.

Can concurrency happen on one core?

Yes. Tasks take turns; when one waits for I/O, another can use the CPU.

Why are backend workloads often I/O-bound?

Requests frequently wait for databases, caches, external services, files, and network responses rather than continuously computing.

Why can event loops handle many sockets efficiently?

They use asynchronous I/O notifications and schedule short callbacks/continuations rather than dedicating a blocked native thread to every waiting connection.

What blocks an event loop?

Long synchronous CPU work or a blocking API call executed on its event-loop thread.

What is a context switch?

Saving the current execution state and restoring another execution's state so the CPU can run it.

Why are native threads expensive?

They require stacks and OS bookkeeping, have creation cost, and can incur context-switch overhead.

What is a goroutine?

A lightweight Go concurrency unit scheduled by the Go runtime over OS threads.

Can async/await code have race conditions?

Yes. Tasks can interleave at await points and observe or update shared state in unsafe orders.

What is a lost update?

Two updates read the same old value and overwrite each other, causing one update to disappear.

How do you prevent races?

Use locks/mutexes, atomics, ownership/message passing such as channels, or database transactions/constraints depending on the shared state.

14. Quick revision sheet

Concurrency

Overlapping progress; can be achieved on one core by interleaving.

Parallelism

Simultaneous execution; requires multiple cores.

I/O-bound

Waiting on network, database, files, or other external I/O.

CPU-bound

Time dominated by computation.

Native thread

OS-managed execution context with stack and scheduler state.

Event loop

Runs short tasks/callbacks; async I/O yields instead of blocking the loop.

async/await

Readable syntax for suspending and resuming asynchronous functions.

Goroutine

Lightweight Go task multiplexed over OS threads by the runtime.

Race condition

Outcome depends on timing/interleaving of concurrent operations.

Mutex

Enforces mutual exclusion for a critical section.

Channel

Message-passing mechanism; can centralize ownership of shared state.

Golden rule

Use concurrency to keep I/O-heavy systems productive; use parallelism to speed up CPU-heavy work.

15. One-page mental model

• A backend request alternates between CPU work and waiting for I/O.

• Concurrency prevents the CPU from sitting idle while one request waits.

• Parallelism uses multiple cores to execute CPU work simultaneously.

• Native threads are OS-managed and carry stack, creation, and context-switch costs.

• Event loops coordinate many I/O operations efficiently, but must not be blocked by long synchronous work.

• Go goroutines are lightweight tasks multiplexed over OS threads by the Go runtime.

• Shared mutable state creates races—even in single-threaded async code when tasks interleave at await points.

• Protect invariants with locks, atomics, channels/ownership, and database transactions as appropriate.